Distributed Multi-Source Data Pipeline for Product Trend Analysis

Jia Liu
CS4265 Big Data Analytics
Kennesaw State University
Spring 2026

***Abstract*—This project presents a distributed big data pipeline for electronics product trend analysis using Apache Spark and Amazon S3. The system integrates multiple heterogeneous datasets including Amazon Electronics reviews, Amazon metadata, Google Trends data, and Common Crawl web text. The pipeline performs distributed ingestion, cleaning, schema normalization, exact and approximate integration, aggregation, and validation at large scale. The final outputs include product-level analytical features such as review statistics, trend scores, and web attention metrics. The project demonstrates important Big Data concepts including distributed processing, distributed storage, ETL pipeline construction, schema alignment, MapReduce-style transformations, and scalable analytical querying. The final system successfully processed over 20 million review records and generated more than 756 thousand integrated analytical outputs.**

I. Executive Summary

Electronic products such as mobile phones, laptops, headphones, and smart devices have become an important part of modern life. Online platforms continuously generate large volumes of product-related data through reviews, search trends, and web discussions. However, processing and integrating these datasets using a single machine is inefficient due to storage limitations, computational bottlenecks, and heterogeneous data formats.

This project builds a distributed multi-source data pipeline using Apache Spark and Amazon S3 to analyze electronics product trends from multiple online datasets. The pipeline integrates structured and unstructured datasets including Amazon Electronics reviews, Amazon metadata, Google Trends data, and Common Crawl web text.

The system performs distributed ingestion, cleaning, normalization, transformation, integration, aggregation, and validation at large scale. Spark is used as the distributed processing framework while Amazon S3 provides scalable distributed object storage. Final analytical outputs are stored in Parquet format to improve query performance and storage efficiency.

The final implementation successfully processed over 20 million Amazon review records and more than 7 million Common Crawl records. The pipeline generated over 756 thousand product-level analytical outputs containing review statistics, trend signals, and web attention metrics.

This project demonstrates important Big Data concepts including distributed storage, distributed processing, ETL pipeline design, schema normalization, scalable analytical systems, and MapReduce-style computation.

II. System Architecture

A. Pipeline Overview

The pipeline follows a distributed batch-processing architecture:

$$\text{S3} \rightarrow \text{Ingestion} \rightarrow \text{Cleaning} \rightarrow \text{Transformation} \rightarrow \text{Integration} \rightarrow \text{Aggregation} \rightarrow \text{Output}$$

The system contains the following stages:

1) Distributed ingestion from Amazon S3
2) Cleaning and preprocessing
3) Schema normalization
4) Exact and approximate integration
5) Product-level aggregation
6) Validation and verification
7) Queryable analytical outputs

B. Technology Stack

TABLE I
Technology Stack

Technology	Purpose
Apache Spark	Distributed processing
Amazon S3	Distributed storage
Spark SQL	Analytical querying
Parquet	Columnar storage format
Python	Pipeline orchestration
AWS CLI	S3 authentication

C. Design Rationale

Apache Spark was selected because the datasets exceeded the practical processing limits of a single machine. Spark provides scalable distributed execution and supports large-scale transformations, joins, and aggregations.

Amazon S3 was selected because it provides scalable distributed object storage and integrates naturally with Spark distributed workflows. S3 also provides fault tolerance and high-capacity storage for large-scale analytical datasets.

Parquet was selected as the final storage format because it improves query performance through columnar storage optimization and compression efficiency.

D. Architecture Diagram

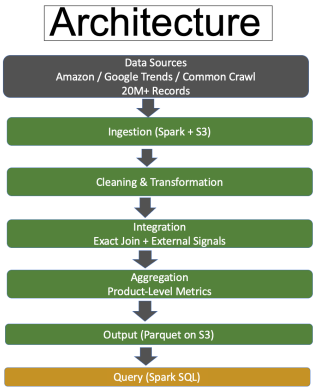

Fig. 1. Distributed pipeline architecture using Apache Spark and Amazon S3.

III. Data Sources and Big Data Characteristics

A. Data Sources

This project integrates four heterogeneous datasets.

TABLE II
Data Sources

Dataset	Format	Size	Purpose
Amazon Reviews	JSON	3.1 GB	User reviews
Amazon Metadata	JSON	1.1 GB	Product metadata
Google Trends	CSV	865 B	Trend signals
Common Crawl	WET/Text	7.87 TB	Web-scale signals

B. Big Data Characteristics

The project demonstrates several important Big Data characteristics.

Volume:

The pipeline processes more than 20 million review records and millions of Common Crawl web text records.

Variety:

The project integrates multiple heterogeneous formats including JSON, CSV, WET text, and Parquet.

Velocity:

The pipeline uses a distributed batch-processing model suitable for large-scale analytical workloads.

IV. Implementation Details

A. Technical Implementation

The project applies several Big Data technologies and distributed system concepts introduced throughout the course.

The implementation focuses on:

- Distributed processing
- Multi-source data integration
- ETL pipeline construction
- Schema normalization
- Large-scale aggregation
- Distributed analytical querying

B. Distributed Processing with Spark

Apache Spark was selected as the primary distributed processing framework because the datasets exceed the computational limits of a single machine.

Spark partitions the datasets and executes distributed transformations in parallel across multiple execution stages.

The implementation applies several MapReduce-style transformations including:

- Distributed filtering
- Parallel transformations
- Shuffle-based joins
- Distributed aggregations
- Feature extraction

Spark SQL and DataFrames were used to simplify distributed analytical processing and schema-based transformations.

C. Amazon S3 Distributed Storage

Amazon S3 was used as scalable distributed object storage for both raw and processed datasets.

The project stores:

- Raw JSON datasets
- CSV trend datasets
- Common Crawl WET files
- Cleaned Parquet outputs
- Final analytical datasets

The use of S3 reflects distributed storage concepts including:

- Scalable object storage
- Distributed persistence
- Fault tolerance
- Large-scale storage abstraction

D. Cleaning and Transformation

Several cleaning and normalization operations were applied during preprocessing.

- Removed invalid records
- Filtered malformed JSON entries
- Validated review ratings
- Standardized categories
- Converted timestamps
- Reduced inconsistent schemas

A reduced schema strategy was applied because the raw JSON datasets contained inconsistent nested structures and missing fields.

Fig. 2. Cleaning and normalization stage.

E. Multi-Source Integration

The pipeline integrates multiple heterogeneous datasets using both exact and approximate integration strategies.

1) Exact Integration: Amazon Reviews and Amazon Metadata were integrated using exact joins based on ASIN product identifiers.

The exact integration stage combines:

- Product identifiers
- Review statistics
- Product titles
- Brand information
- Category information

2) Approximate Integration: Google Trends and Common Crawl datasets do not contain direct relational keys compatible with Amazon datasets.

Approximate integration was performed using:

- Electronics topic mapping
- Category alignment
- Trend score aggregation
- Web mention extraction

The final integrated dataset combines structured product analytics with external web-scale signals.

F. Aggregation

The aggregation stage generates product-level analytical signals including:

- Review count
- Average rating
- Trend scores
- Web mention counts
- Review length metrics

The final outputs are stored as Parquet datasets in Amazon S3.

G. Course Concepts Applied

This project applies multiple concepts introduced throughout the semester.

1) Data Integration and Schema Alignment: The pipeline integrates heterogeneous datasets with different formats and structures including JSON, CSV, WET text, and Parquet. Schema normalization techniques were used to align inconsistent structures into a unified analytical representation.

2) MapReduce Paradigm and Distributed Processing: The project follows the MapReduce processing model discussed during the course.

Examples include:

- Map-style extraction and transformation
- Distributed shuffle joins
- Reduce-style aggregations
- Parallel partition processing

3) Parallel Processing Patterns: The pipeline demonstrates parallel processing concepts by distributing ingestion, cleaning, transformation, and aggregation operations across Spark partitions.

Parallel execution significantly improves scalability for large-scale datasets.

4) ETL Pipeline Design: The project implements a complete ETL pipeline including:

- Extract: Distributed ingestion from Amazon S3
- Transform: Cleaning, normalization, integration, and aggregation
- Load: Final Parquet outputs stored in Amazon S3

5) Pipeline Optimization: Several optimization strategies were applied:

- Reduced schema extraction
- Distributed partitioning
- Columnar Parquet storage
- Spark SQL optimization
- Early filtering of invalid records

These optimizations reduced unnecessary shuffle operations and improved distributed execution efficiency.

6) Data Validation and Quality Assurance: The project also applies data validation and quality assurance concepts introduced later in the semester.

Validation procedures included:

- Schema verification
- Record count validation
- Read-back verification
- Sample query validation
- Rating range validation

V. Validation and Results

A. Dataset Statistics

The final pipeline successfully processed the following datasets.

TABLE III
Dataset Statistics

Dataset	Records
Amazon Reviews	20,994,353
Amazon Metadata	786,445
Google Trends	53
Common Crawl	7,006,623
Integrated Dataset	21,835,272
Product Signals	756,420
Final Output	756,420

B. Cleaning Results

TABLE IV
CLEANING RESULTS

Dataset	Before	After	Removed
Amazon Reviews	20,994,353	20,984,629	9,724
Amazon Metadata	786,445	786,445	0
Google Trends	53	53	0
Common Crawl	7,006,623	2,565,925	4,440,698

C. Runtime Performance

TABLE V
PIPELINE RUNTIME

Pipeline Stage	Runtime
Ingestion	269.99 seconds
Cleaning & Transformation	494.14 seconds
Integration & Aggregation	3014.17 seconds
Storage	5313.79 seconds
Verification	652.42 seconds
Total Runtime	9763.16 seconds

The full distributed pipeline completed successfully in approximately 2.7 hours.

D. Data Quality Validation

Several validation procedures were performed.

- Schema verification
- Record count verification
- Read-back validation
- Rating range validation
- Sample query validation

Validation confirmed that the final dataset preserved product identifiers, metadata, review statistics, and external signals correctly after distributed aggregation.

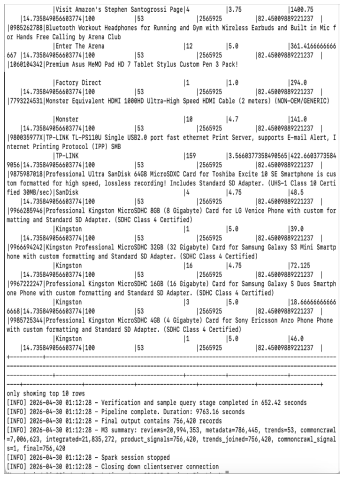

Fig. 3. Final output schema verification and sample query results.

E. Edge Case Handling

The pipeline includes handling for several edge cases.

TABLE VI
EDGE CASE HANDLING

Edge Case	Handling
Invalid JSON	Filtered during parsing
Missing fields	Removed during cleaning
Duplicate records	Removed during filtering
Unexpected schema fields	Reduced schema applied
Empty datasets	Aggregation safely skipped

VI. CHALLENGES ENCOUNTERED

Several technical challenges were encountered during implementation.

A. Large-Scale Data Processing

The datasets exceeded the storage and processing limits of a single machine.

Solution:

Spark distributed processing and Amazon S3 distributed storage were used to enable scalable parallel execution.

B. Heterogeneous Data Formats

The pipeline needed to process JSON, CSV, and WET text simultaneously.

Solution:

Modular ingestion functions and schema normalization strategies were implemented for each data source.

C. Schema Inconsistencies

The Amazon JSON datasets contained inconsistent nested structures and missing fields.

Solution:

A reduced schema approach was applied to improve stability and simplify distributed processing.

D. Approximate Integration Complexity

External datasets lacked direct relational keys compatible with Amazon datasets.

Solution:

Topic-level approximate matching and external signal generation were used.

E. Spark Logging and Debugging

Several Spark logging and runtime formatting issues occurred during development.

Solution:

Logging was standardized using Python f-strings and structured runtime tracking for each pipeline stage.

VII. Known Limitations

Although the pipeline successfully demonstrates distributed multi-source integration, several limitations remain.

- Common Crawl integration currently uses keyword-level matching.
- Google Trends data granularity is limited.
- Large distributed joins remain computationally expensive.
- The current implementation supports batch processing only.
- Approximate integration may not fully capture semantic relationships.

VIII. Reflection and Future Improvements

This project provided practical experience with distributed systems and large-scale data engineering.

The project improved understanding of:

- Distributed processing using Apache Spark
- Scalable storage using Amazon S3
- ETL pipeline construction
- Schema normalization
- Distributed joins and aggregations
- Spark performance optimization
- Technical documentation using LaTeX

Several future improvements are possible.

- Improved semantic matching for external signals
- Better Spark partition optimization
- Real-time streaming support
- Advanced NLP processing for Common Crawl text
- Improved trend prediction models

IX. Conclusion

This project successfully implemented a distributed multi-source data pipeline for electronics product trend analysis using Apache Spark and Amazon S3.

The system demonstrated scalable ingestion, cleaning, transformation, integration, aggregation, and validation across multiple heterogeneous datasets. The final pipeline processed over 20 million review records and generated more than 756 thousand analytical outputs.

The project demonstrates important Big Data concepts including distributed processing, distributed storage, ETL pipeline design, schema normalization, scalable analytics systems, and distributed analytical querying. The final implementation is fully functional, validated, and portfolio-ready.